C++ STL Containers — Hinglish Revision Notes

Scope: `vector` se `map` tak ke commonly used STL containers, DSA aur Competitive Programming (CP) focus ke saath.

References: cppreference — Containers · cplusplus.com — STL containers

> **How to use:** Pehle container ka purpose samjho, phir member-function signatures/overloads revise karo, aur example compile karke dekho. Code snippets C++11+ ko target karte hain; kuch notes mein C++17+ clearly marked hai.

---

0. Common concepts

- **Iterator:** Container ke element/position ko refer karta hai. `begin()` first element par, `end()` last element ke *baad* hota hai.
- **Range `[first, last)`:** `first` include hota hai, `last` include nahi hota.
- **`size()` vs `capacity()`:** `size()` actual elements; `capacity()` vector kitne elements reallocation ke bina rakh sakta hai.
- **`insert()` return:** Aksar inserted element(s) ke first element ka iterator; associative containers mein return type overload ke hisaab se alag ho sakta hai.
- **`erase()` return:** Aam taur par erased range ke baad wale element ka iterator; container-specific overloads alag ho sakte hain.
- **Invalidation:** Insert/erase ke baad kuch iterators/references invalid ho sakte hain. Vector mein reallocation hone par existing element iterators/references/pointers invalidate ho sakte hain.
- **Complexity:** Neeche typical standard complexity di hai; exact overload aur position ke hisaab se farq ho sakta hai.

Container quick choice

Container	Kya store karta hai	Order / access	Typical use
`vector<T>`	Sequence	Contiguous; index O(1)	Arrays, sorting, CP default
`list<T>`	Sequence	Doubly linked; no indexing	Known iterator position par insert/erase
`deque<T>`	Sequence	Index O(1); dono ends par efficient	Front aur back dono se operations
`stack<T>`	LIFO adaptor	Sirf `top()`	Brackets, monotonic stack

Container	Kya store karta hai	Order / access	Typical use
`queue<T>`	FIFO adaptor	`front()` / `back()`	BFS, scheduling
`priority_queue<T>`	Heap adaptor	Highest priority `top()`	Top-K, heap problems
`set<T>`	Unique keys	Sorted	Unique sorted values
`multiset<T>`	Duplicate keys allowed	Sorted	Frequency/order statistics style tasks
`map<K,V>`	Unique key-value pairs	Keys sorted	Key lookup, frequency/value mapping

---

1. `vector`

Header: `<vector>`
Concept: Dynamic array; elements contiguous memory mein hote hain. Index access O(1), end par `push_back` amortized O(1).

Member functions — arguments and return

```
/*
.begin()                    -> iterator to first element
.end()                      -> iterator past last element
.rbegin() / .rend()         -> reverse iterators
.cbegin() / .cend()         -> const iterators
.crbegin() / .crend()       -> const reverse iterators

.size()                     -> number of elements; O(1)
.empty()                    -> bool; empty hai ya nahi
.capacity()                 -> current capacity
.max_size()                 -> theoretical max element count

.front()                    -> reference to first element
.back()                     -> reference to last element
[index] / operator[](n)     -> element reference; bounds check nahi
.at(n)                      -> element reference; invalid index par
                               std::out_of_range throw karta hai
.data()                     -> pointer to underlying contiguous array
```

```
.push_back(value)           -> end par copy/move element; void
.emplace_back(args...)      -> end par in-place construct;
                               C++17+ mein inserted element ka reference
.pop_back()                 -> last element remove; void

.insert(pos, value)         -> pos se pehle value; iterator return
.insert(pos, count, value)  -> pos se pehle count copies; iterator return
.insert(pos, first, last)   -> iterator range [first,last) insert
.insert(pos, init_list)     -> initializer_list insert
.emplace(pos, args...)      -> pos par in-place construct; iterator return

.erase(pos)                 -> ek element remove; next iterator return
.erase(first, last)         -> [first,last) remove; next iterator return
.clear()                    -> all elements remove; size 0
.resize(n)                  -> size n; badhne par default-insert
.resize(n, value)           -> size n; badhne par value insert
.reserve(n)                 -> capacity kam-se-kam n ki request; size nahi badalta
.shrink_to_fit()            -> capacity reduce karne ki non-binding request

.assign(count, value)       -> count copies se replace
.assign(first, last)        -> range se replace
.assign(init_list)          -> initializer_list se replace
.swap(other)                -> contents exchange; void
*/
```

Example: vector operations

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
using namespace std;

int main() {
    vector<int> v = {30, 10, 20};

    v.push_back(40);
    v.emplace_back(50);
    v.insert(v.begin() + 1, 15);       // index 1 se pehle
    v.insert(v.begin(), 2, 5);         // start par two 5s
    v.erase(v.begin());                // first element
    v.erase(v.begin(), v.begin() + 1); // range [first,last)
    v.resize(5, 99);
    v.reserve(20);                     // size unchanged
    sort(v.begin(), v.end());
```

```
    cout << "size=" << v.size() << '\n';
    cout << "front=" << v.front() << ", back=" << v.back() << '\n';
    cout << "index 0=" << v[0] << ", at(1)=" << v.at(1) << '\n';
    cout << "sum=" << accumulate(v.begin(), v.end(), 0) << '\n';

    auto it = find(v.begin(), v.end(), 99);
    cout << (it != v.end() ? "99 found" : "99 not found") << '\n';

    for (auto x : v) cout << x << ' ';
    cout << '\n';

    v.pop_back();
    vector<int> other = {7, 8};
    v.swap(other);
    other.clear();

    cout << "empty after clear: " << other.empty() << '\n';
}
```

Revision points - `reserve(n)` size change nahi karta; `resize(n)` karta hai. - `v[i]` bounds-check nahi karta; `v.at(i)` karta hai. - `insert/erase` ke baad elements shift ho sakte hain: middle operation O(n). - `sort`, `find`, `reverse`, `lower_bound`, `upper_bound` algorithms `<algorithm>` mein hain, vector member functions nahi. - `accumulate` `<numeric>` mein hai. - `lower_bound` / `upper_bound` ke liye range sorted honi chahiye.

Common complexity: index/front/back O(1); `push_back` amortized O(1); middle insert/erase O(n); sorting O(n log n).

---

2. `list`

Header: `<list>`
Concept: Doubly linked list. Random indexing (`l[2]`) supported nahi. Iterator se position mil jaaye to insert/erase efficient hote hain.

Member functions

```
/*
.begin() / .end()           -> iterators
.rbegin() / .rend()         -> reverse iterators
.cbegin() / .cend()         -> const iterators
.size()                     -> element count
.empty()                    -> bool
.max_size()                 -> theoretical max size
.front() / .back()          -> first/last element reference
```

```
.push_front(value)           -> beginning par insert
.emplace_front(args...)      -> beginning par in-place construct
.pop_front()                 -> first element remove
.push_back(value)            -> end par insert
.emplace_back(args...)       -> end par in-place construct
.pop_back()                  -> last element remove

.insert(pos, value)          -> pos se pehle insert; iterator return
.insert(pos, count, value)   -> count copies insert
.insert(pos, first, last)    -> iterator range insert
.insert(pos, init_list)      -> initializer_list insert
.emplace(pos, args...)       -> pos par in-place construct

.erase(pos)                  -> one element erase; next iterator return
.erase(first, last)          -> [first,last) erase
.clear()                     -> all elements remove
.resize(n)                   -> size n
.resize(n, value)            -> size n; growth par value use
.assign(count, value)        -> count copies se replace
.assign(first, last)         -> range se replace
.assign(init_list)           -> initializer_list se replace
.swap(other)                 -> contents exchange

.remove(value)               -> value ke saare matching elements remove
.remove_if(predicate)        -> predicate true wale elements remove
.unique()                    -> adjacent equal duplicates remove
.unique(binary_predicate)    -> adjacent elements jo predicate match karein remove
.sort()                      -> ascending order
.sort(compare)               -> comparator ke according sort
.reverse()                   -> order reverse
.merge(other)                -> sorted lists merge; dono sorted honi chahiye
.merge(other, compare)       -> comparator ke according merge
.splice(pos, other)           -> other ki saari nodes pos se pehle move
.splice(pos, other, it)       -> other ka element it move
.splice(pos, other, first,last)-> other ka [first,last) move
*/
```

Note: `list::sort()` aur `list::merge()` list ke member functions hain. Inke liye `std::sort(l.begin(), l.end())` use nahi kar sakte, kyunki `std::sort` random-access iterators maangta hai.

Example

```
#include <iostream>
#include <list>
```

```cpp
using namespace std;

int main() {
    list<int> l = {20, 30, 40};

    l.push_front(10);
    l.push_back(50);
    l.emplace_front(5);
    l.emplace_back(60);

    auto it = l.begin();
    advance(it, 2);              // iterator ko index 2 tak le jao
    l.insert(it, 15);             // position se pehle
    l.erase(l.begin());           // first remove
    l.remove(30);                 // all 30s remove

    l.push_back(40);
    l.unique();                   // sirf adjacent duplicates remove
    l.sort();
    l.reverse();

    cout << "front=" << l.front() << ", back=" << l.back() << '\n';
    for (int x : l) cout << x << ' ';
    cout << '\n';

    list<int> other = {1, 2, 3};
    l.splice(l.end(), other);     // other ki nodes l ke end par move
    cout << "other empty=" << other.empty() << '\n';

    l.pop_front();
    l.pop_back();
    l.clear();
    cout << "size=" << l.size() << '\n';
}
```

Revision points - `advance(it, index)` iterator ko aage le jaata hai; `<iterator>` include karo. - `list` mein direct index access nahi. - `remove(value)` values remove karta hai; `erase(iterator)` iterator position remove karta hai. - `unique()` sirf **consecutive/adjacent** duplicates remove karta hai. Pehle `sort()` karne se equal values adjacent ho sakti hain. - Insert/splice ke liye iterator position use hoti hai; `splice` nodes transfer karta hai.

Typical complexity: known iterator par insert/erase O(1); search O(n); `size()` O(1); sort O(n log n).

---

3. deque

Header: `<deque>`
Concept: Double-ended queue. Front aur back dono par efficient insertion/removal; random access bhi available.

Member functions

```
/*
.begin() / .end()              -> iterators
.rbegin() / .rend()            -> reverse iterators
.cbegin() / .cend()            -> const iterators
.size()                        -> element count
.empty()                       -> bool
.max_size()                    -> theoretical max size
.capacity()                    -> deque mein nahi hota

.front() / .back()             -> first/last element reference
[index] / operator[](n)        -> index access, no bounds checking
.at(n)                         -> checked index access; invalid index throws

.push_back(value)              -> end par insert
.emplace_back(args...)         -> end par in-place construct
.pop_back()                    -> last remove
.push_front(value)             -> beginning par insert
.emplace_front(args...)        -> beginning par in-place construct
.pop_front()                   -> first remove

.insert(pos, value)            -> pos se pehle insert
.insert(pos, count, value)     -> count copies insert
.insert(pos, first, last)     -> range insert
.insert(pos, init_list)        -> initializer_list insert
.emplace(pos, args...)         -> pos par in-place construct

.erase(pos)                    -> one element remove
.erase(first, last)             -> [first,last) remove
.clear()                       -> all elements remove
.resize(n)                     -> size change
.resize(n, value)              -> growth par value insert
.shrink_to_fit()               -> capacity-style control API nahi; deque mein nahi
.assign(count, value)          -> count copies se replace
.assign(first, last)           -> range se replace
.assign(init_list)             -> initializer_list se replace
.swap(other)                   -> contents exchange
*/
```

Example

```cpp
#include <iostream>
#include <deque>
using namespace std;

int main() {
    deque<int> dq = {20, 30};

    dq.push_front(10);
    dq.push_back(40);
    dq.emplace_front(5);
    dq.emplace_back(50);
    dq.insert(dq.begin() + 2, 15);
    dq.erase(dq.begin() + 1);

    cout << "index 1=" << dq[1] << '\n';
    cout << "front=" << dq.front() << ", back=" << dq.back() << '\n';

    dq.pop_front();
    dq.pop_back();

    for (int x : dq) cout << x << ' ';
    cout << '\n';

    deque<int> other = {7, 8};
    dq.swap(other);
    dq.clear();
    cout << "empty=" << dq.empty() << '\n';
}
```

Revision points - `deque` mein `dq[i]` aur `dq.at(i)` valid hain. - `deque` mein `capacity()` / `reserve()` nahi hote. - Front aur back insertion/removal efficient hote hain. - Middle insertion/erasure generally O(n); operations iterators ko invalidate kar sakte hain.

---

4. `stack`

Header: `<stack>`
Concept: LIFO — Last In, First Out. Container adaptor hai, general-purpose iterable container nahi.

Member functions

```
/*
.empty()                -> bool; stack empty hai ya nahi
.size()                 -> number of elements
.top()                  -> top element ka reference; peek jaisa
.push(value)            -> top par element add; void
.emplace(args...)       -> top par in-place construct; void
.pop()                  -> top element remove; void, value return nahi karta
.swap(other)            -> two stacks ka content exchange
*/
```

Important: C++ STL `stack` mein `peek()` nahi; `top()` uska equivalent hai. `bottom()`, indexing, iterators available nahi hote.

Example

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> st;

    st.push(10);
    st.push(20);
    st.emplace(30);

    cout << "top=" << st.top() << '\n'; // 30
    cout << "size=" << st.size() << '\n';

    st.pop();                           // 30 remove
    cout << "new top=" << st.top() << '\n'; // 20

    stack<int> other;
    other.push(99);
    st.swap(other);

    while (!st.empty()) {
        cout << st.top() << ' ';
        st.pop();
    }
    cout << '\n';
}
```

Revision points - `top()` read karta hai, remove nahi karta. - `pop()` value return nahi karta; pehle `top()` read karo, phir `pop()`. - Empty stack par `top()`

ya `pop()` call mat karo. - Default underlying container `deque<T>` hota hai; `vector<T>` ya `list<T>` bhi specify kar sakte ho if requirements satisfy hon.

---

5. queue

Header: `<queue>`
Concept: FIFO — First In, First Out.

Member functions

```
/*
.empty()                -> bool
.size()                 -> number of elements
.front()                -> first element reference; next to pop
.back()                 -> last element reference; most recently added
.push(value)            -> rear/end par element add; void
.emplace(args...)       -> rear par in-place construct; void
.pop()                  -> front element remove; void
.swap(other)            -> contents exchange
*/
```

Example

```cpp
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;

    q.push(10);
    q.push(20);
    q.emplace(30);

    cout << "front=" << q.front() << '\n'; // 10
    cout << "back=" << q.back() << '\n';   // 30

    q.pop();
    cout << "new front=" << q.front() << '\n'; // 20

    while (!q.empty()) {
        cout << q.front() << ' ';
        q.pop();
    }
```

```
    cout << '\n';
}
```

Revision points - `queue` FIFO follow karti hai. - `front()` aur `back()` dekhte hain; `pop()` front remove karta hai. - `queue` mein iterators ya indexing expose nahi hote. - Empty queue par `front()`, `back()` ya `pop()` call mat karo.

---

6. priority_queue

Header: `<queue>`
Concept: Heap-based container adaptor. Default `priority_queue<int>` max-heap hai; sabse bada element `top()` par hota hai.

Member functions

```
/*
.empty()                -> bool
.size()                 -> number of elements
.top()                  -> highest-priority element reference
.push(value)            -> element add; void
.emplace(args...)       -> element in-place construct; void
.pop()                  -> top element remove; void
.swap(other)            -> contents exchange
*/
```

Max-heap vs min-heap

```
priority_queue<int> maxHeap; // largest top
priority_queue<int, vector<int>, greater<int>> minHeap; // smallest top
```

Example

```
#include <iostream>
#include <queue>
#include <vector>
#include <functional>
using namespace std;

int main() {
    priority_queue<int> pq;
    pq.push(10);
    pq.push(50);
    pq.emplace(20);

    cout << "max top=" << pq.top() << '\n'; // 50
```

```
    pq.pop();
    cout << "new top=" << pq.top() << '\n'; // 20

    priority_queue<int, vector<int>, greater<int>> minPQ;
    minPQ.push(10);
    minPQ.push(50);
    minPQ.push(20);
    cout << "min top=" << minPQ.top() << '\n'; // 10

    while (!minPQ.empty()) {
        cout << minPQ.top() << ' ';
        minPQ.pop();
    }
    cout << '\n';
}
```

Revision points - Default is **max-heap**; `greater<T>` se min-heap banta hai. - `top()` value return nahi karta by value; top element ka const reference deta hai. - `push` / `pop` O(log n), `top` O(1). - `priority_queue` ke paas `begin()`, `end()`, indexing ya `clear()` member function nahi.

---

7. set

Header: `<set>`
Concept: Unique keys, sorted order. Default ascending order; common operations O(log n).

Member functions

```
/*
.begin() / .end()              -> sorted order ke iterators
.rbegin() / .rend()            -> reverse iterators
.cbegin() / .cend()            -> const iterators
.size()                        -> number of unique elements
.empty()                       -> bool
.max_size()                    -> theoretical maximum size

.insert(value)                 -> value insert;
                                  pair<iterator,bool> return (single value)
.insert(hint, value)           -> hinted insert; iterator return
.insert(first, last)           -> range insert; void
.insert(init_list)             -> initializer_list insert; void
.emplace(args...)              -> construct + insert; pair<iterator,bool>
.emplace_hint(hint, args...)    -> hinted construct/insert; iterator
```

```
.erase(key)                    -> key ke elements remove; count return (0 or 1)
.erase(pos)                    -> iterator position remove; next iterator return
.erase(first, last)             -> range remove; next iterator return
.clear()                       -> all elements remove
.swap(other)                   -> contents exchange

.find(key)                     -> iterator, absent ho to end()
.count(key)                    -> 0 ya 1
.contains(key)                 -> bool; C++20+
.lower_bound(key)              -> first element >= key
.upper_bound(key)              -> first element > key
.equal_range(key)              -> pair of lower_bound and upper_bound iterators
.key_comp()                    -> key comparator copy
.value_comp()                  -> value comparator copy
*/
```

Example

```cpp
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<int> s = {30, 10, 20, 10}; // duplicate 10 removed

    s.insert(40);
    s.emplace(25);

    cout << "size=" << s.size() << '\n';

    auto it = s.find(20);
    cout << (it != s.end() ? "20 found" : "20 not found") << '\n';

    cout << "count(10)=" << s.count(10) << '\n';
    cout << "lower_bound(22)=" << *s.lower_bound(22) << '\n';
    cout << "upper_bound(20)=" << *s.upper_bound(20) << '\n';

    s.erase(30);

    for (int x : s) cout << x << ' ';
    cout << '\n';

    s.clear();
    cout << "empty=" << s.empty() << '\n';
}
```

Revision points - Duplicate values store nahi hote. - Elements sorted order mein iterate hote hain. - Set ke elements ko iterator se modify nahi kar sakte, kyunki modification ordering tod sakta hai. - `find`, `insert`, `erase(key)`, `lower_bound`, `upper_bound`: O(log n). - `contains()` C++20 se available hai; older compiler mein `find(key) != end()` use karo.

---

8. `multiset`

Header: `<set>`
Concept: Sorted keys, duplicates allowed. Frequency maintain karne aur ordered duplicates ke liye useful.

Member functions

`multiset` ke almost saare member functions `set` jaise hi hote hain:

```
/*
.begin() / .end() / .rbegin() / .rend()
.cbegin() / .cend() / .crbegin() / .crend()
.size() / .empty() / .max_size()

.insert(value)                -> iterator return; duplicates allowed
.insert(hint, value)          -> iterator return
.insert(first, last)          -> range insert
.insert(init_list)            -> initializer_list insert
.emplace(args...)             -> iterator return
.emplace_hint(hint, args...)  -> iterator return

.erase(key)                   -> number of matching keys removed
.erase(pos)                   -> next iterator
.erase(first, last)            -> next iterator
.clear()                      -> all elements remove
.swap(other)                  -> contents exchange

.find(key)                    -> matching element iterator or end()
.count(key)                   -> duplicate occurrences count
.contains(key)                -> bool; C++20+
.lower_bound(key)             -> first element >= key
.upper_bound(key)             -> first element > key
.equal_range(key)             -> matching-key range
.key_comp() / .value_comp()   -> comparator copy
*/
```

Example

```cpp
#include <iostream>
#include <set>
using namespace std;

int main() {
    multiset<int> ms = {10, 20, 10, 30, 10};

    cout << "count(10)=" << ms.count(10) << '\n'; // 3

    ms.insert(20);
    ms.erase(ms.find(10)); // sirf ek 10 remove

    cout << "after erase one 10: ";
    for (int x : ms) cout << x << ' ';
    cout << '\n';

    ms.erase(10); // key ke saare 10 remove
    cout << "after erase all 10: ";
    for (int x : ms) cout << x << ' ';
    cout << '\n';
}
```

Critical difference - `ms.erase(10)` → key `10` ki **saari occurrences** erase karta hai. - `ms.erase(ms.find(10))` → sirf ek occurrence erase karta hai. - `count(key)` duplicates ki total occurrences batata hai; multiset mein O(log n + count) tak cost ho sakti hai.

---

9. `map`

Header: `<map>`
Concept: Unique keys ke saath key-value pairs; keys sorted order mein rehti hain. Typical lookup/insert/erase O(log n).

Member functions and overloads

```cpp
/*
.begin() / .end()             -> iterator over pair<const Key, T>
.rbegin() / .rend()           -> reverse iterators
.cbegin() / .cend()           -> const iterators
.size()                       -> key-value pair count
.empty()                      -> bool
.max_size()                   -> theoretical max pair count
```

```
operator[key]                 -> value reference; missing key ho to
                                 default-inserted mapped value banata hai
.at(key)                      -> mapped value reference; missing key par
                                 std::out_of_range throw karta hai

.insert(value_type)           -> pair<iterator,bool>
.insert(hint, value_type)     -> iterator
.insert(first, last)          -> range insert
.insert(init_list)            -> initializer_list insert
.insert_or_assign(key, value) -> insert ya existing mapped value assign;
                                 pair<iterator,bool>
.insert_or_assign(hint,key,value) -> hinted version; iterator

.emplace(args...)             -> construct + insert; pair<iterator,bool>
.emplace_hint(hint,args...)   -> hinted version; iterator
.try_emplace(key,args...)     -> key missing ho to mapped value construct;
                                 pair<iterator,bool>
.try_emplace(hint,key,args...) -> hinted version; iterator

.erase(key)                   -> 0 or 1 return
.erase(pos)                   -> next iterator return
.erase(first,last)             -> next iterator return
.clear()                      -> all pairs remove
.swap(other)                  -> contents exchange

.find(key)                    -> iterator or end()
.count(key)                   -> 0 or 1
.contains(key)                -> bool; C++20+
.lower_bound(key)             -> first key >= given key
.upper_bound(key)             -> first key > given key
.equal_range(key)             -> lower/upper bound pair
.key_comp()                   -> key comparator copy
.value_comp()                 -> pair comparator copy
*/
```

map ke important access differences

```
map<int, string> mp;

mp[1] = "Dinesh";       // insert/update
mp[2] = "Rahul";

cout << mp[1];          // Dinesh
cout << mp[3];          // missing key insert ho jayegi, string empty

// cout << mp.at(4);    // key 4 missing: throws out_of_range
```

```cpp
auto it = mp.find(2);
if (it != mp.end())
    cout << it->first << " " << it->second;
```

- `mp[key]` key missing ho to entry create karta hai; read-only lookup ke liye ye side effect yaad rakho.
- `mp.at(key)` missing key par exception throw karta hai; entry create nahi karta.
- `find()` se check kar sakte ho ki key present hai ya nahi.
- Map mein `pair<const Key, T>` hota hai; iterator se key modify nahi kar sakte.

Example

```cpp
#include <iostream>
#include <map>
#include <string>
using namespace std;

int main() {
    map<int, string> mp;

    mp[3] = "C";
    mp[1] = "A";
    mp.insert({2, "B"});
    mp.emplace(4, "D");
    mp.insert_or_assign(2, "Bee");
    mp.try_emplace(5, "E");

    cout << "size=" << mp.size() << '\n';
    cout << "mp[1]=" << mp[1] << '\n';
    cout << "at(2)=" << mp.at(2) << '\n';
    cout << "count(3)=" << mp.count(3) << '\n';
    cout << "contains(4)=" << mp.contains(4) << '\n'; // C++20

    auto it = mp.find(3);
    if (it != mp.end())
        cout << it->first << " -> " << it->second << '\n';

    cout << "lower_bound(2): " << mp.lower_bound(2)->first << '\n';
    cout << "upper_bound(2): " << mp.upper_bound(2)->first << '\n';

    mp.erase(1);

    for (const auto& [key, value] : mp) { // structured bindings: C++17
```

```
        cout << key << " -> " << value << '\n';
    }

    mp.clear();
    cout << "empty=" << mp.empty() << '\n';
}
```

Revision points - `map` keys unique aur sorted hoti hain. - `mp[key]` insert/update kar sakta hai; `find()` sirf search karta hai. - `insert()` existing key ka mapped value overwrite nahi karta. - `insert_or_assign()` existing key ki value update ya missing key insert karta hai (C++17+). - `try_emplace()` key already present ho to mapped value ko construct/insert nahi karta (C++17+). - `contains()` C++20+; `insert_or_assign()` aur `try_emplace()` C++17+.

---

10. Fast revision: container selection

Requirement	Choose
Fast indexing + sorting	`vector`
Front/back insertions + indexing	`deque`
Frequent insert/erase at known iterator	`list`
Last in, first out	`stack`
First in, first out	`queue`
Repeatedly get largest/smallest	`priority_queue`
Sorted unique values	`set`
Sorted values with duplicates	`multiset`
Sorted unique keys with values	`map`

Common mistakes checklist

1. `stack`, `queue`, `priority_queue` mein `begin()` / `end()` nahi hote.
2. `vector` aur `deque` index support karte hain; `list` nahi.
3. `priority_queue<int>` default **max-heap** hai.
4. `set` / `map` unique keys/values ke liye; duplicates chahiye to `multiset` / `multimap`.
5. `map[key]` missing key create kar sakta hai.
6. `erase(key)` multiset mein same key ki saari occurrences remove karta hai.
7. `unique()` (list member ya `std::unique` algorithm) general unsorted duplicates ko automatically sab jagah remove nahi karta; algorithm consecutive duplicates compact karta hai.
8. `std::sort()` ko random-access iterators chahiye; `list.sort()` use karo.
9. `lower_bound` / `upper_bound` sorted ranges ya ordered associative containers par use karo.

10. Empty container par `front()`, `back()`, `top()`, `pop()` se pehle emptiness check karo.

Official reference links

- cppreference — Containers library
- cppreference — `vector`
- cppreference — `list`
- cppreference — `deque`
- cppreference — `stack`
- cppreference — `queue`
- cppreference — `priority_queue`
- cppreference — `set`
- cppreference — `multiset`
- cppreference — `map`
- cplusplus.com — STL containers